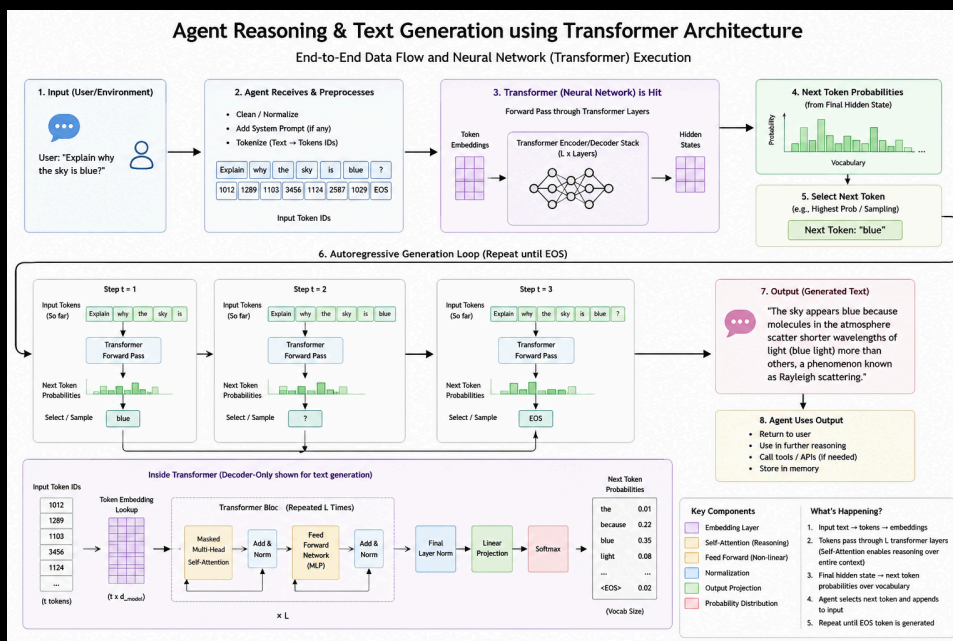


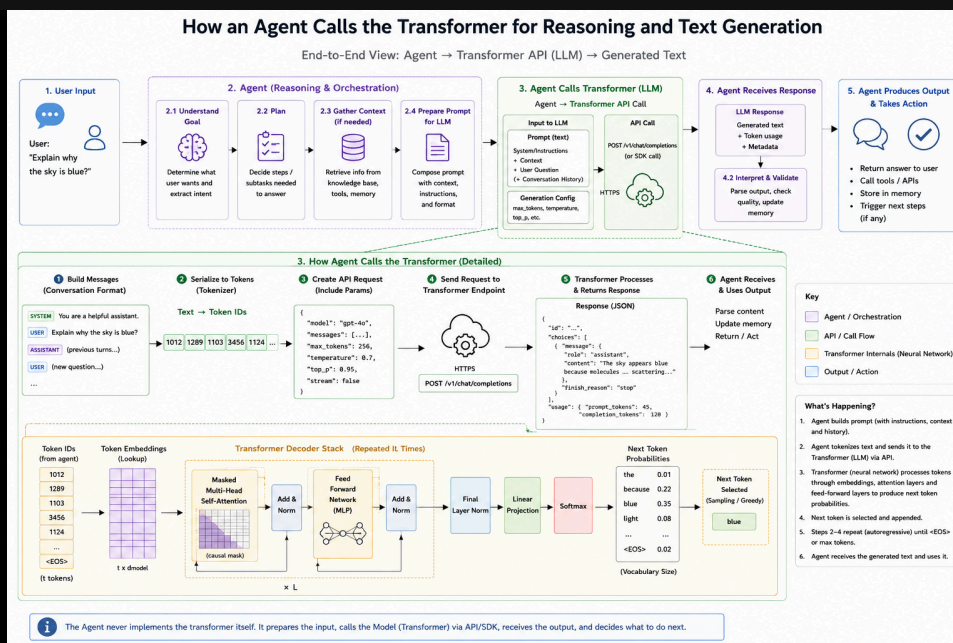
This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Suppose agent need to reason and generate text so it uses transformer architecture. Show a diagram how data flows and neural network is hit, show with diagram



In the above diagram how agent calls the transformer. Show this.



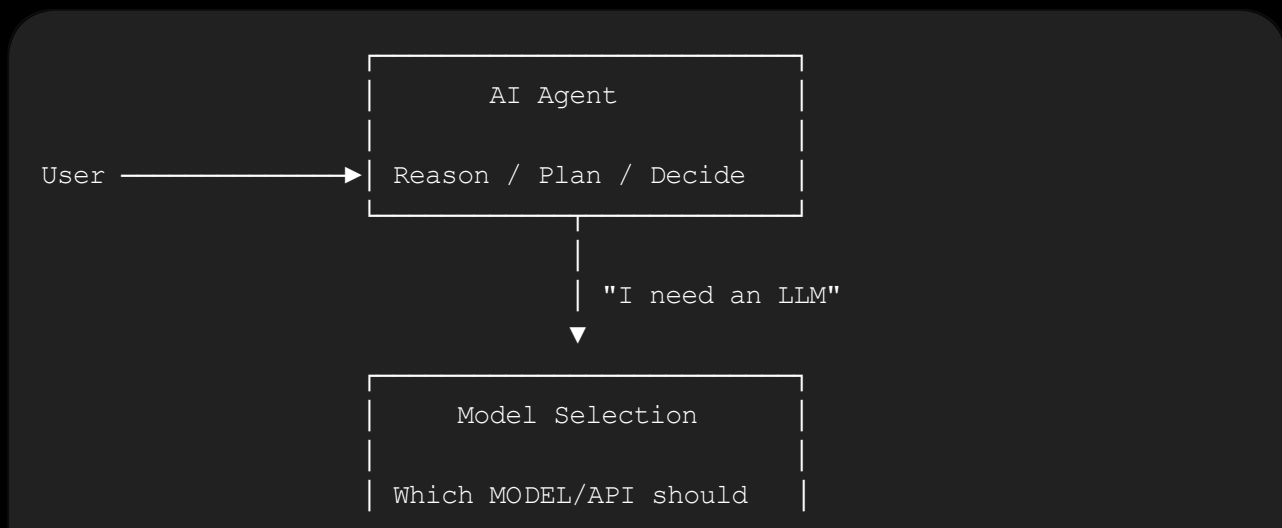
Neural network architectures can be transformer, Encoder only network, Encoder Decoder, CNN. how when agent calls LLM how it is decided which neural network architecture will be considered

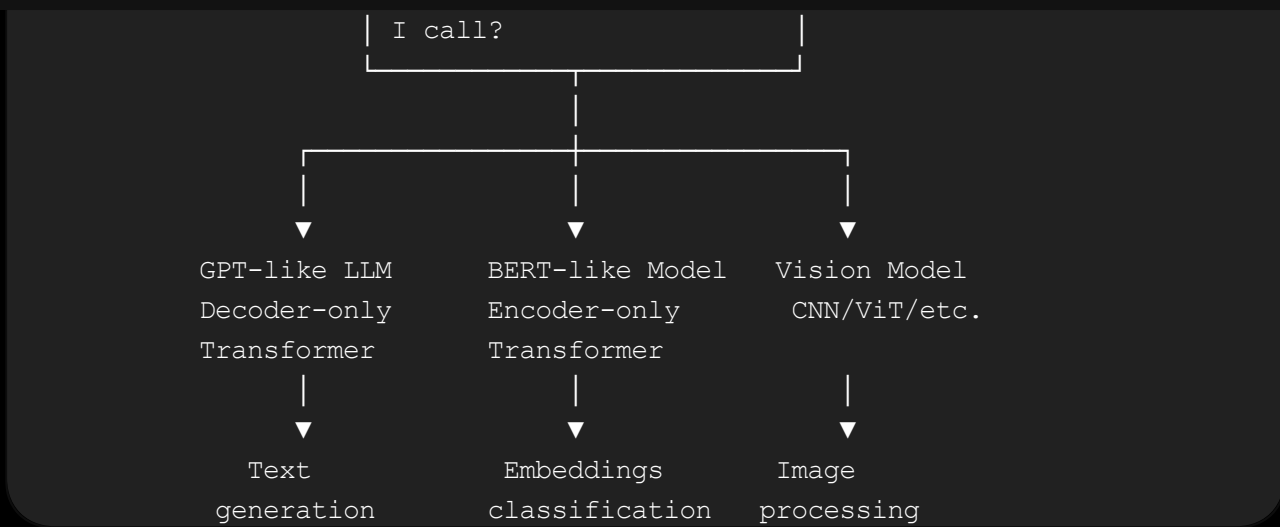
The key idea is: **the agent usually does not decide whether to use Transformer, CNN, Encoder-only, or Encoder-Decoder. The model/API it calls has already been built with a particular neural-network architecture.**

Think of the agent as an **orchestrator**, not as the component that selects the neural-network architecture.

1. The architecture is chosen when the model is built

A simplified flow is:





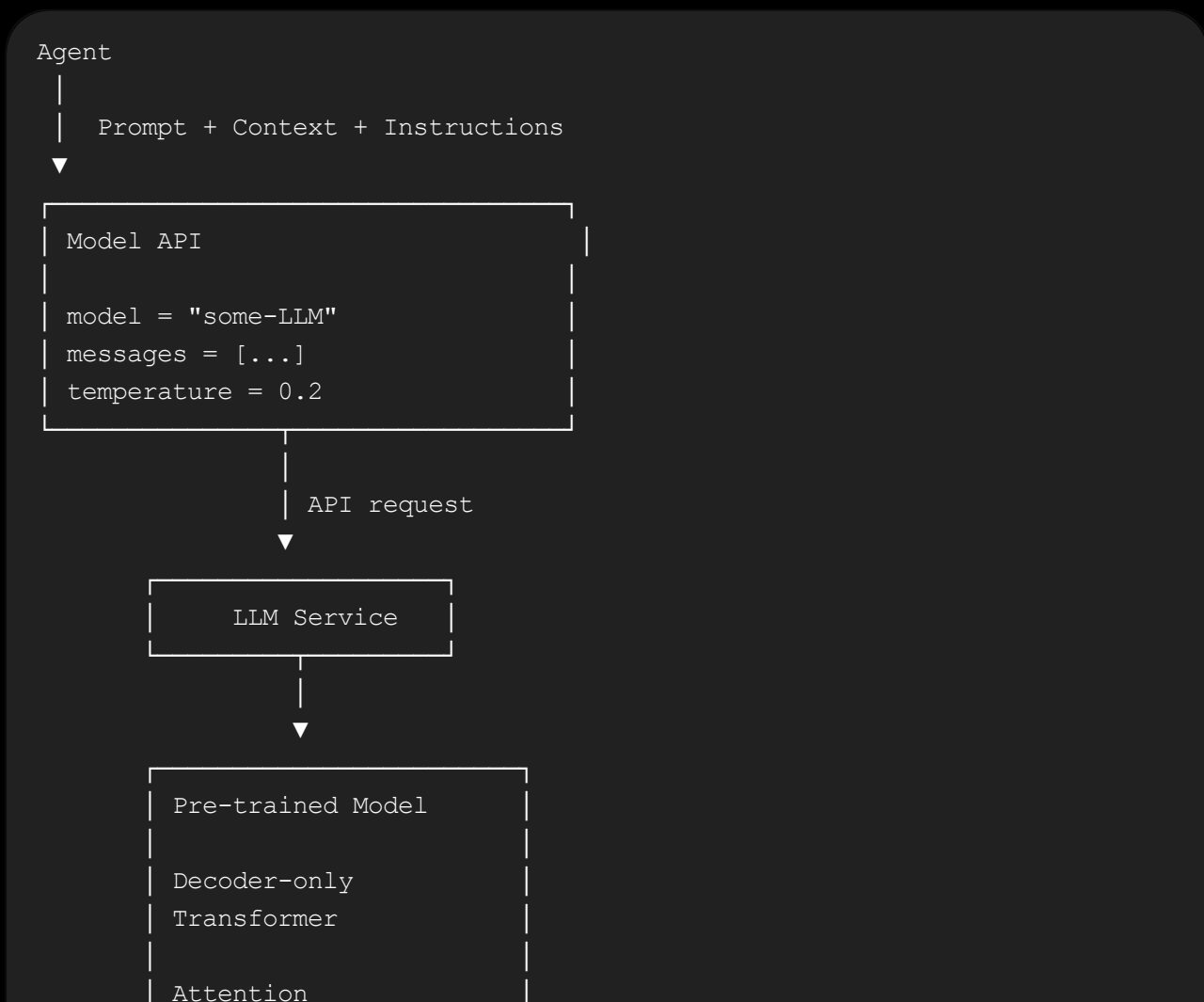
The agent might select **a model**, but the model's architecture is normally fixed.

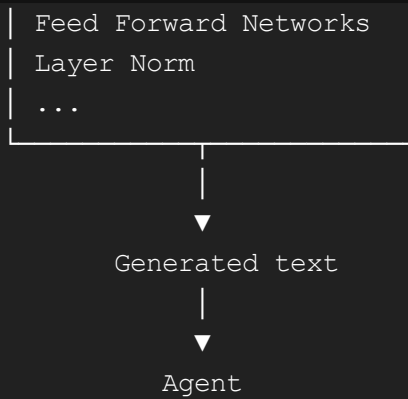
2. What happens when an agent calls an LLM?

Suppose your agent receives:

"Analyze this insurance claim and explain whether it should be approved."

The agent might construct something like:





The important distinction is:

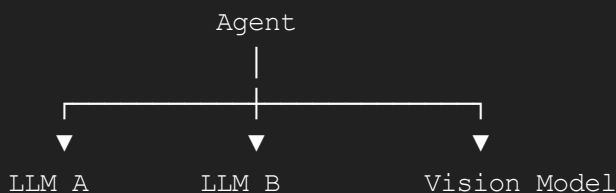
The agent sends data to the model. It doesn't normally construct or choose the internal neural-network layers at runtime.

3. So where does the architecture decision happen?

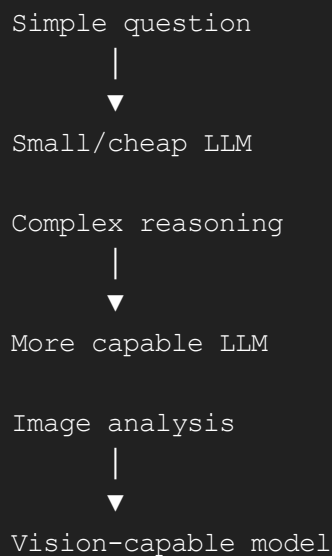
There are actually **three different decisions** that are easy to confuse.

Decision A — Which AI model?

This can be an agent decision.



For example:

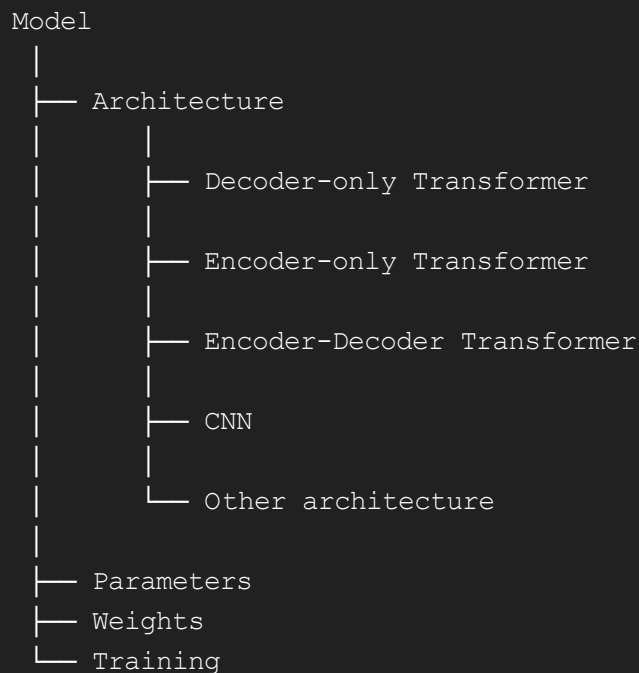


The agent/orchestration layer can select the appropriate **model endpoint**.

Decision B — What architecture does that model use?

This is generally decided **before deployment/training**.

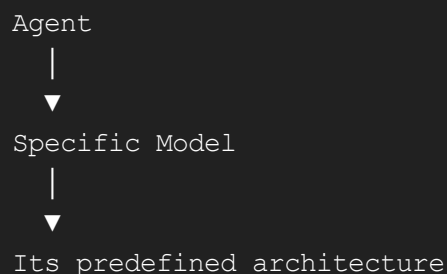
For example:



The agent doesn't normally say:

"Today I'll use 12 Transformer layers
and tomorrow I'll use a CNN."

Instead:



4. Why are there different architectures?

Because they are optimized for different types of problems.

Architecture	Typical purpose	Example task
Decoder-only Transformer	Generate sequences	Chat, reasoning, code generation
Encoder-only Transformer	Understand/represent input	Classification, embeddings
Encoder–Decoder Transformer	Transform one sequence into another	Translation, summarization
CNN	Spatial/local feature extraction	Image recognition
Vision Transformer (ViT)	Image understanding	Image classification
Multimodal architecture	Combine modalities	Image + text understanding

So the question shouldn't really be:

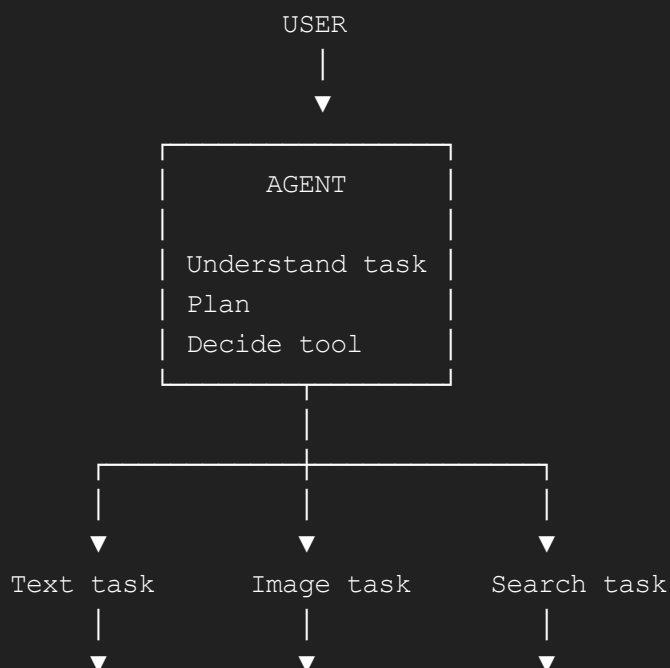
"How does the agent decide which architecture to use?"

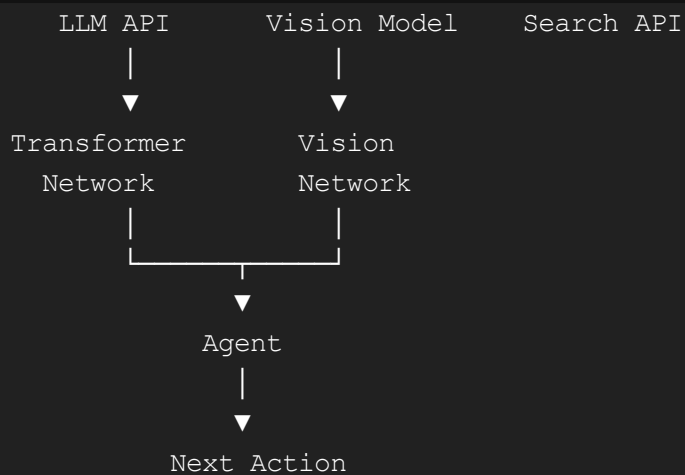
It is more accurately:

"How does the agent decide which model/tool to invoke for the task?"

5. Agent architecture makes this much clearer

Consider a sophisticated agent:





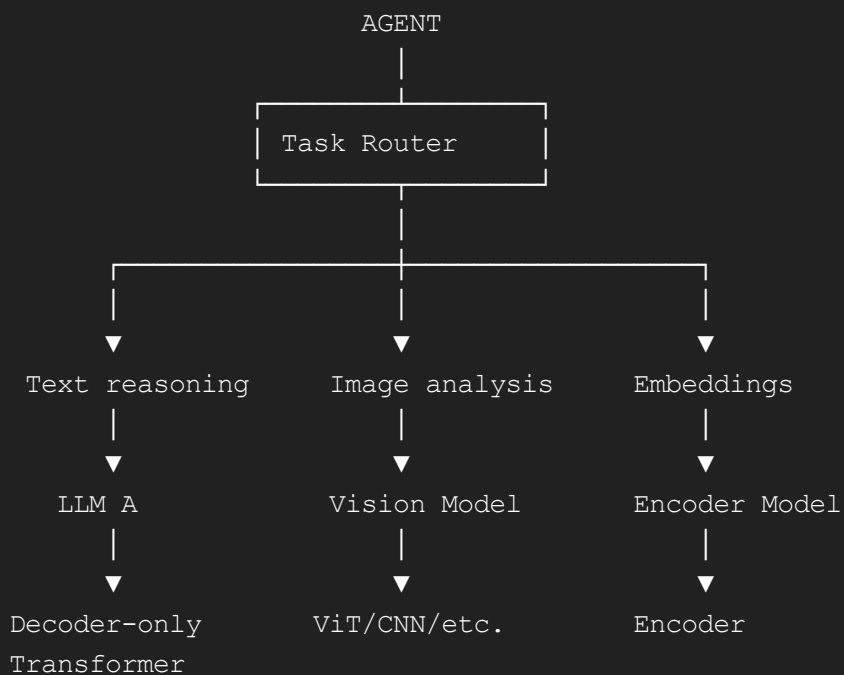
The agent chooses the capability/model.

The model determines the neural architecture.

6. But there is an important exception

In modern AI systems, an agent can be designed with **multiple models**.

For example:



Here the agent can dynamically choose which model to call.

But notice the distinction:

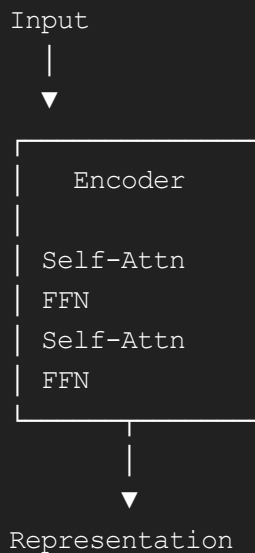
Agent chooses:
MODEL

Model has:
ARCHITECTURE

7. What about Encoder vs Decoder?

This is particularly important for understanding LLMs.

Encoder-only



Good for:

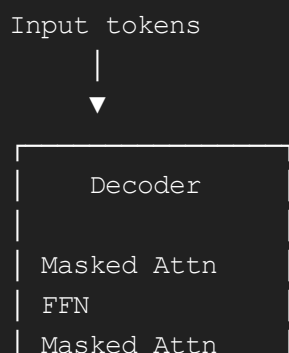
"What does this text mean?"

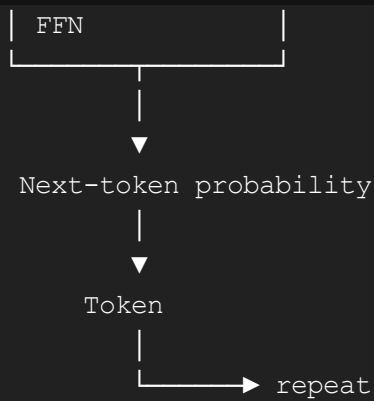
"Is this review positive?"

"Find similar documents."

"Generate an embedding."

Decoder-only





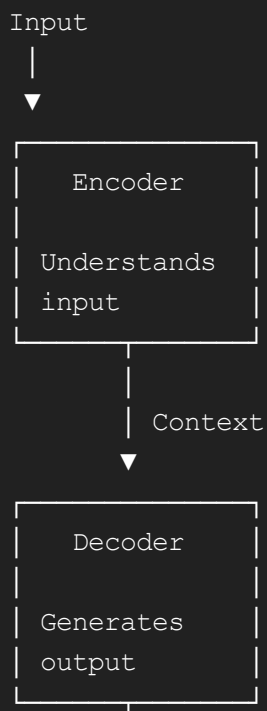
This is particularly suitable for:

```
Prompt
↓
Generate token
↓
Generate next token
↓
Generate next token
↓
...
```

This is why decoder-only Transformers are a natural fit for **chatbots**, **coding assistants**, **reasoning systems** and **generative agents**.

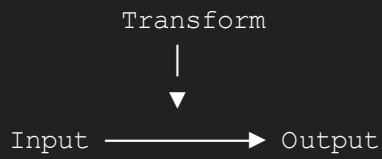
8. Encoder–Decoder

Here the architecture has two major parts:





This is useful when the task is essentially:

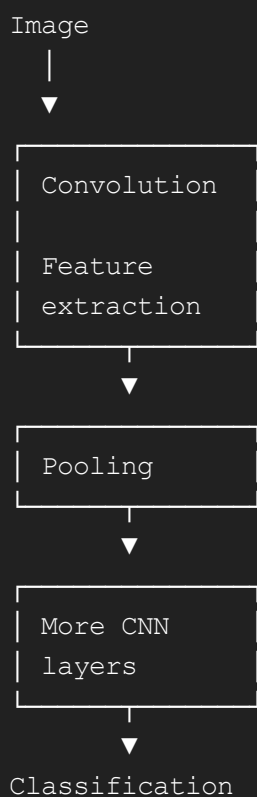


For example:

English → French
Long text → Summary
Question → Answer

9. CNN is different

A CNN is generally associated with spatial data such as images.



So if an agent receives:

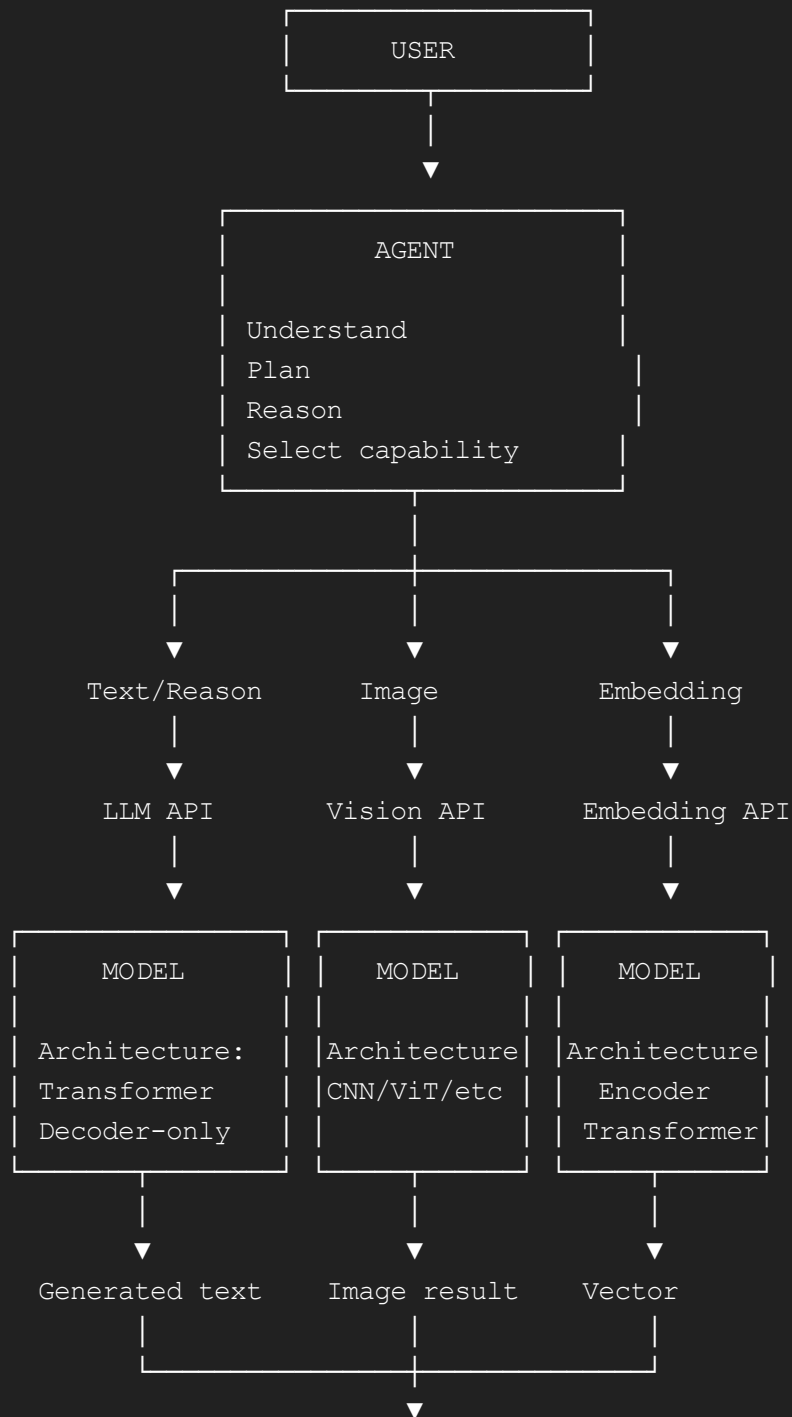
"Look at this damaged-car photograph and determine whether the bumper is damaged."

the agent may route the task to a **vision-capable model**.

The vision model might use a Transformer, CNN, or another architecture depending on how that model was designed.

10. The most important conceptual diagram

I would visualize the whole system like this:



The mental model to remember

Agent ≠ Neural Network.

An agent is primarily an **orchestration/control layer** that can:

- understand the task,
- maintain state/context,
- plan,
- choose a model,
- call tools,
- call APIs,
- inspect results,
- decide the next step.

The **LLM/model** contains the neural network that performs the learned computation.

And the **model's architecture**—Transformer decoder, encoder, encoder-decoder, CNN, etc.—is an implementation/design characteristic of that model.

So the hierarchy is:

Agent → selects/calls Model → Model runs its predefined Neural Network Architecture → produces output → Agent decides what to do next.

This distinction is fundamental when designing **Agentic AI architectures**.

Add example of LLM names/types with specific neural network architecture

Yes. Adding **real model examples** makes the distinction much easier. One important caveat: for some proprietary models, the exact internal architecture is not publicly disclosed, so we should only label architectures that the provider/research paper has actually described.

Model → Architecture → Typical Agent Usage

Model / Model family	Neural-network architecture	Typical use in an agent
GPT / GPT-style models	Transformer, generally decoder-only /	Reasoning, planning, text/code generation, tool calling

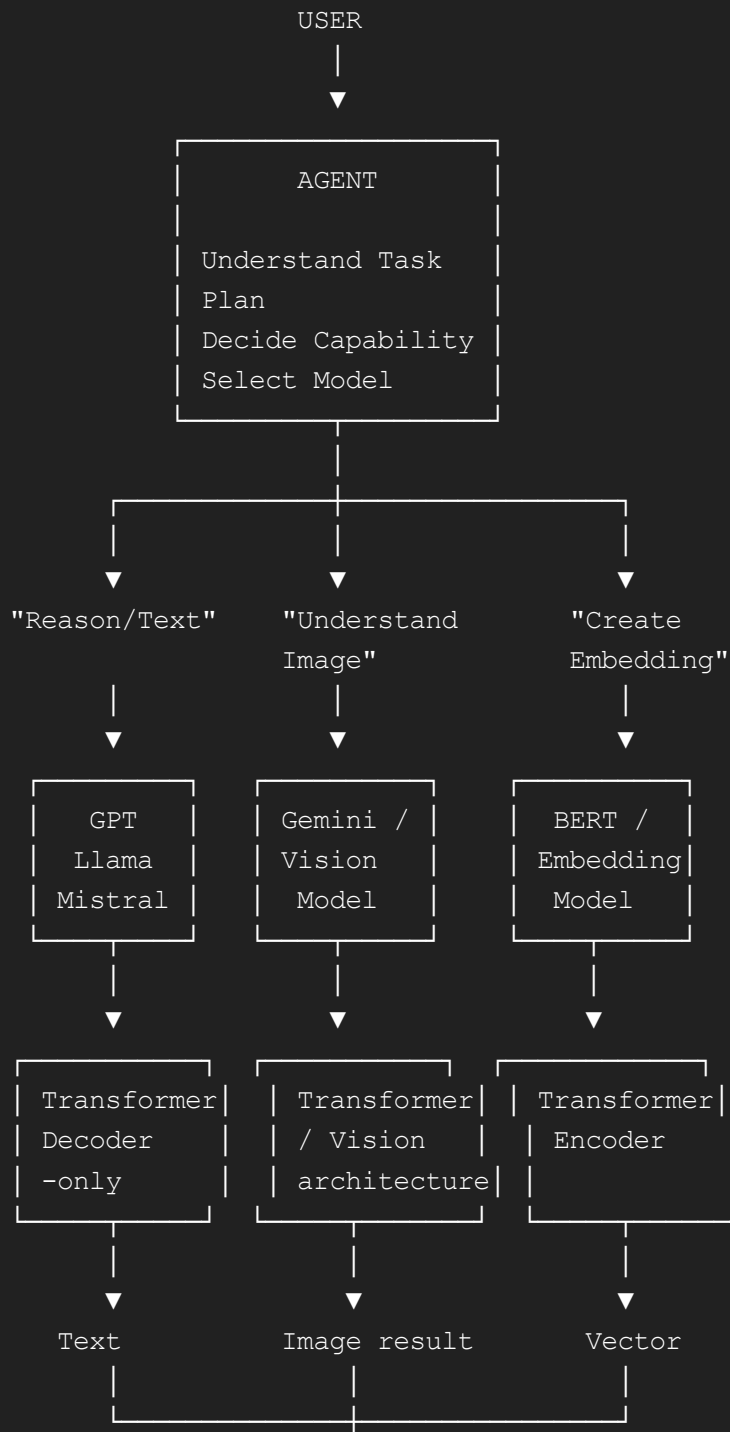
Model / Model family	Neural-network architecture	Typical use in an agent
	autoregressive for GPT-style LMs	
OpenAI gpt-oss-20b / 120b	Transformer + Mixture-of-Experts (MoE)	Reasoning, tool use, agentic workflows
Google Gemini	Transformer decoder-based , with multimodal components	Reasoning, text, image/audio/video understanding
Meta Llama	Decoder-only Transformer	Chat, reasoning, coding, agents
Mistral / Mixtral	Transformer; Mixtral uses Mixture-of-Experts	Efficient reasoning, coding, agents
BERT	Encoder-only Transformer	Classification, embeddings, semantic understanding
T5	Encoder–Decoder Transformer	Translation, summarization, text-to-text transformation
Whisper	Encoder–Decoder Transformer	Speech → text, translation
CNN models	Convolutional Neural Network	Image/object/visual feature processing
Vision Transformer (ViT)	Transformer encoder	Image understanding/classification

For example, Google's original Gemini technical report explicitly describes Gemini as building on **Transformer decoders**, while OpenAI's current open-weight `gpt-oss` models are described as Transformers using **Mixture-of-Experts**.  +1

BERT is explicitly an encoder-based Transformer, while T5 is an encoder-decoder Transformer.  +1 Whisper is also explicitly documented by OpenAI as an encoder-decoder Transformer. 

Put this into your Agent architecture

The complete picture becomes:

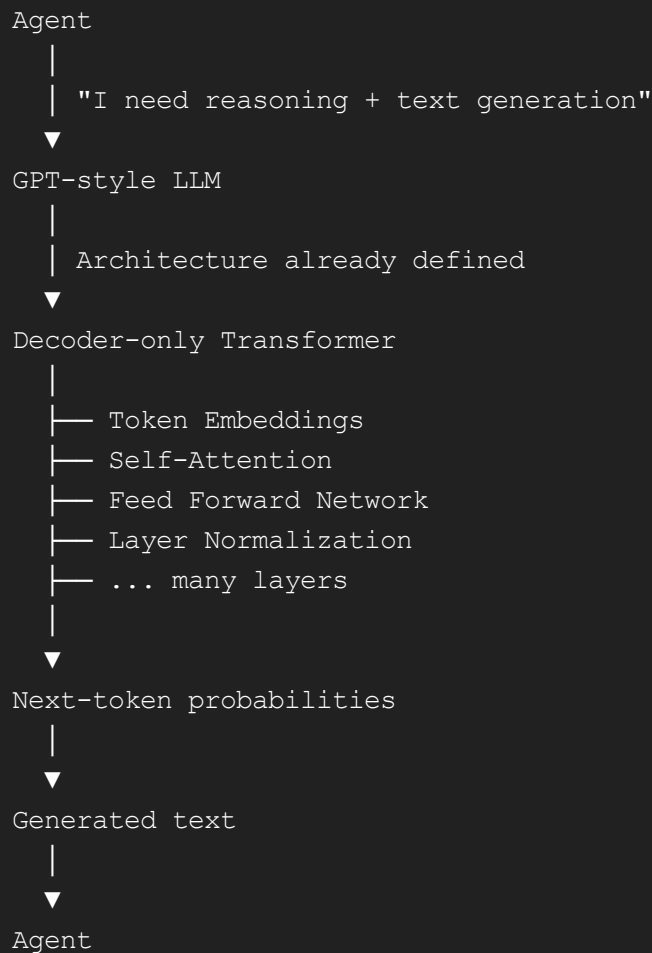


The key relationship

Think of it as:

Agent chooses the MODEL → MODEL has an ARCHITECTURE → architecture processes the data.

For example:



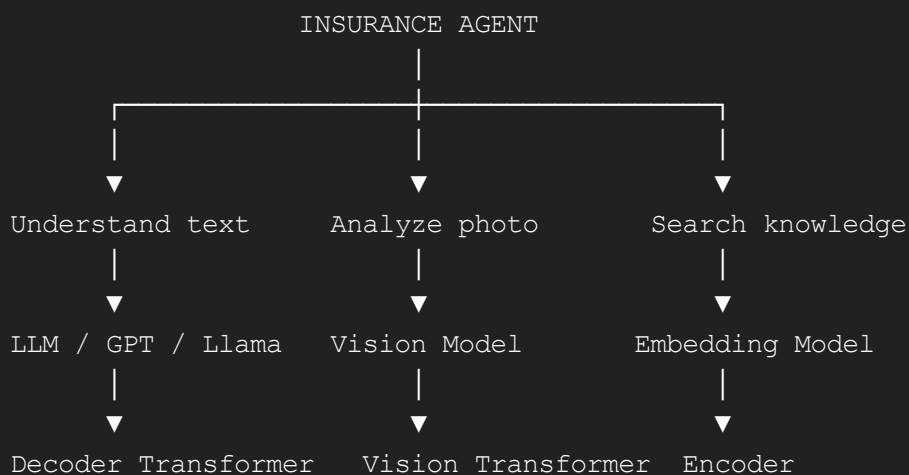
The agent **doesn't dynamically construct the Transformer**.

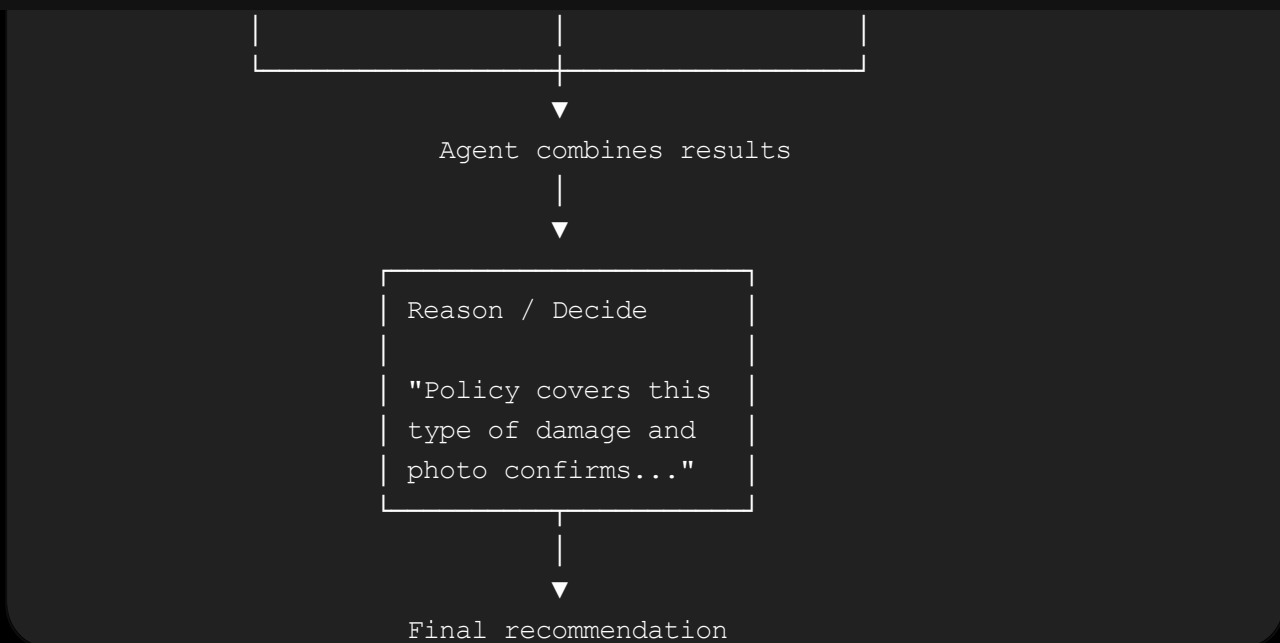
And this is where Agentic AI becomes interesting

Suppose an agent receives:

"Analyze this insurance claim, look at the attached accident photo, search our policy database, and recommend whether to approve it."

Now the agent may orchestrate **multiple model capabilities**:





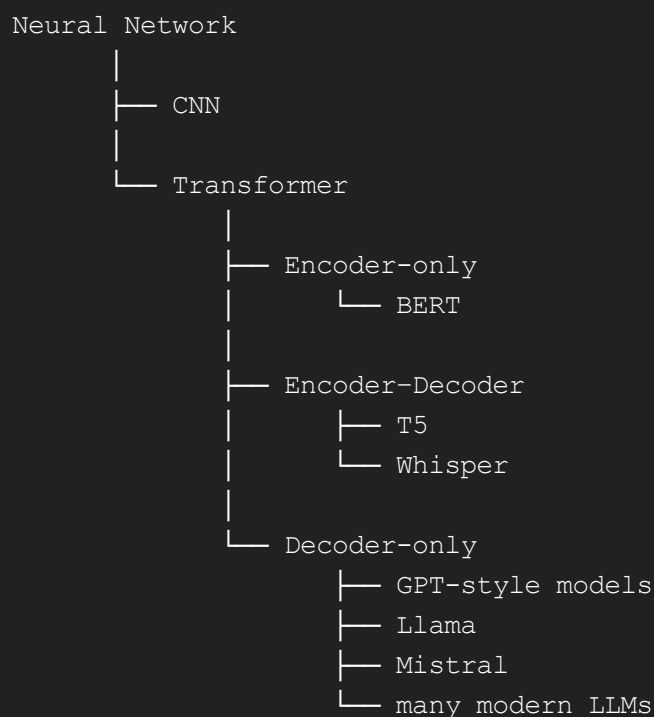
This is a much more accurate mental model of a **modern AI agent** than saying "the agent is a Transformer."

One more important distinction

Transformer is an architecture.

GPT, Llama, Gemini, BERT, T5, etc. are model families/models built using particular architectures.

So:



And MoE is not a completely separate architecture family in the same sense; it is a technique used inside some Transformer models to route tokens through a subset of expert

networks. OpenAI's gpt-oss models are an example.  OpenAI

If you're building your **Agentic AI architecture knowledge**, the next useful diagram is "**Agent → Model Router → LLM / Embedding / Vision / Speech / Tool → Neural Network → Output → Agent**", because that shows exactly where **model selection, API calls, tokenization, and the actual neural-network forward pass** occur.

What are the type of neural network architectures

Neural-network architectures can be grouped into several major families. **Transformer is only one family**; CNN, RNN, GAN, autoencoder, etc. are other architectural patterns.

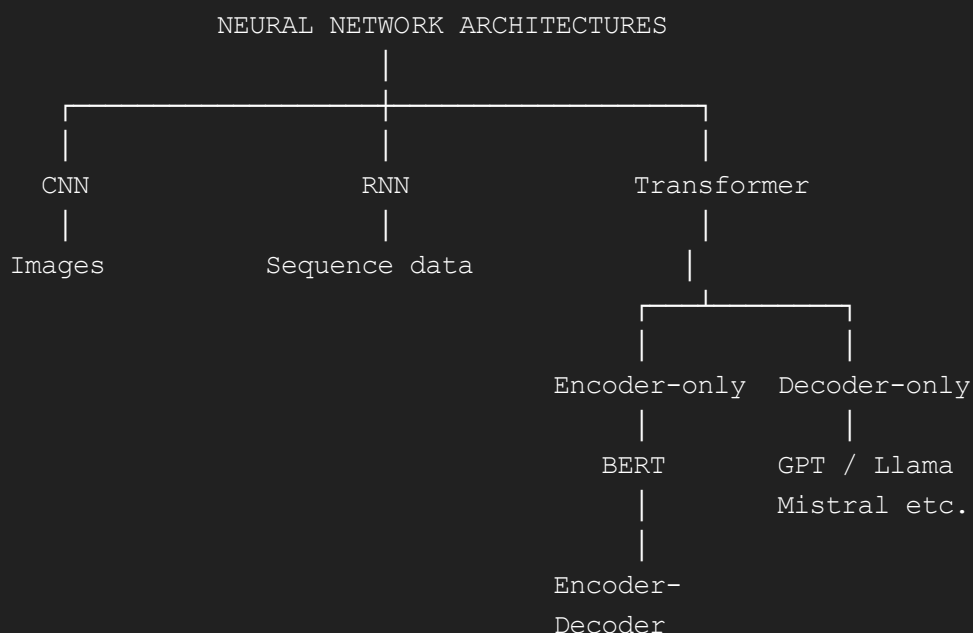
Major neural-network architectures

Architecture	Main idea	Typical data/task	Examples
Feed-Forward Neural Network (FNN/MLP)	Data flows forward through fully connected layers	Tabular data, classification, regression	MLP
CNN (Convolutional Neural Network)	Learns local spatial patterns using convolution	Images, video, spatial data	ResNet, VGG, EfficientNet
RNN (Recurrent Neural Network)	Maintains information from previous sequence steps	Sequential/time-series data	Vanilla RNN
LSTM	RNN with memory gates	Long sequences, time series, speech	LSTM
GRU	Simplified gated RNN	Sequential data	GRU
Transformer	Uses attention to model relationships between tokens/elements	Text, code, images, audio, multimodal data	GPT, BERT, T5, Llama

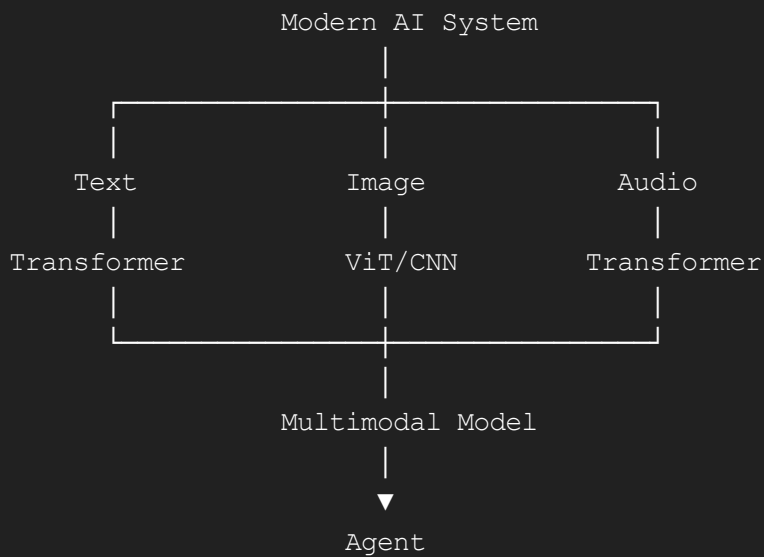
Architecture	Main idea	Typical data/task	Examples
Autoencoder	Encoder compresses data; decoder reconstructs it	Representation learning, anomaly detection	VAE, standard AE
GAN	Generator competes with discriminator	Image/data generation	DCGAN, StyleGAN
Graph Neural Network (GNN)	Learns relationships between graph nodes/edges	Networks, recommendations, molecules	GCN, GAT
Diffusion models	Learns to reverse a noise-generation process	Image/audio/video generation	Stable Diffusion, Imagen
Mixture-of-Experts (MoE)	Routes different inputs/tokens to specialized expert networks	Large-scale LLMs	Mixtral, GPT-oss
Siamese / Twin networks	Compares representations from two inputs	Similarity/search/verification	Face/text similarity models

Transformer itself has important variants

When discussing LLMs, this hierarchy is particularly important:

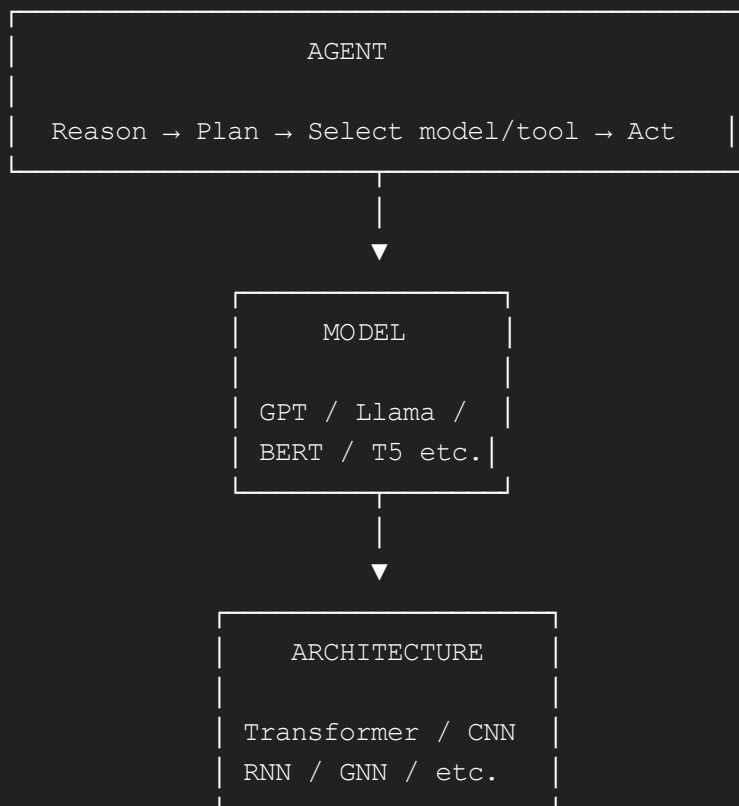


And modern systems can combine architectures:



For Agentic AI, remember this distinction

There are **three different levels**:



So **GPT is not itself a neural-network architecture**. GPT is a model family built around a **Transformer decoder architecture**. Similarly, **BERT is a model based on the Transformer encoder architecture**, while CNN is itself an architectural family.

For your Agentic AI learning, the most important architectures to understand deeply are **MLP → CNN → RNN/LSTM/GRU → Transformer → Encoder/Decoder variants → MoE → Multimodal architectures**.

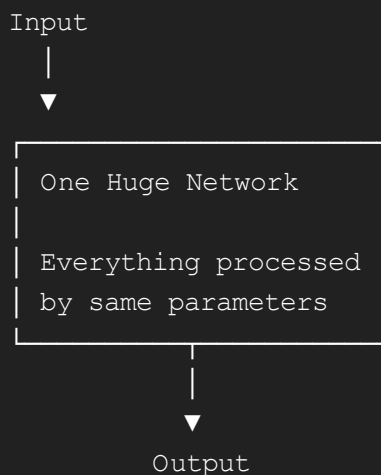
What are mixture of networks

You likely mean **Mixture of Experts (MoE)**, not "Mixture of Networks."

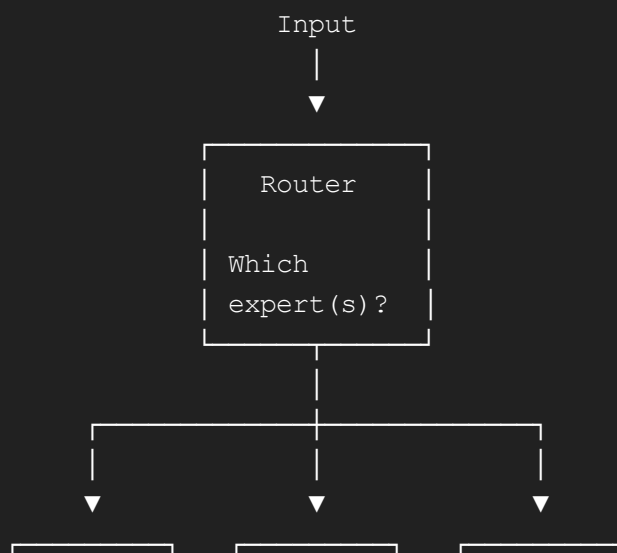
Mixture of Experts is an architecture/technique where a large neural network contains multiple specialized sub-networks ("experts"), and a routing mechanism decides which expert(s) should process each input/token.

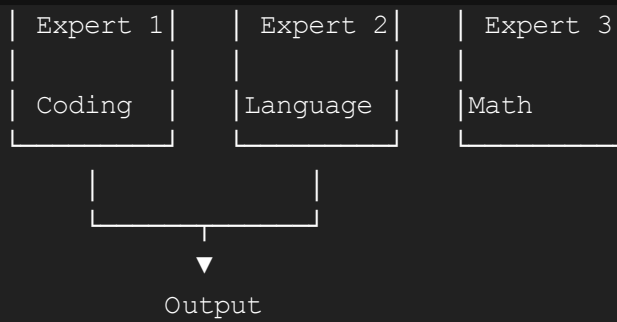
1. Basic idea

Instead of having one huge neural network process everything:



MoE does this:





The important point is that **not every expert needs to run for every input**.

2. Why do this?

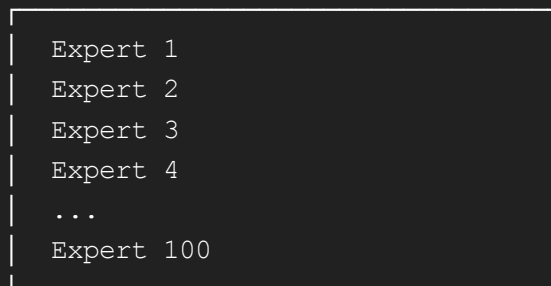
Suppose an LLM has:

1 trillion total parameters

A conventional dense model might use essentially the whole network for every token.

An MoE model might have:

Total parameters



Router selects

Only a few experts activated

For a particular token, the router might select:

Token: "Azure Function deployment"

Router

|
├ Expert 17 → 0.62
├ Expert 42 → 0.31
├ Expert 73 → 0.02
└ others → very low

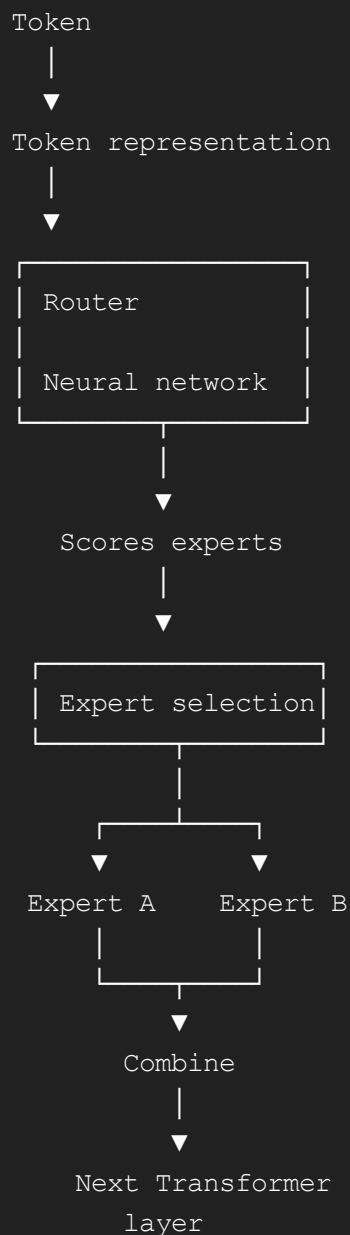
So experts 17 and 42 might process that token.

This gives the model a **large total parameter capacity without requiring all parameters to be activated for every token**.

3. Where does the Router come from?

The router is itself a small neural-network component.

Conceptually:



The router doesn't normally understand:

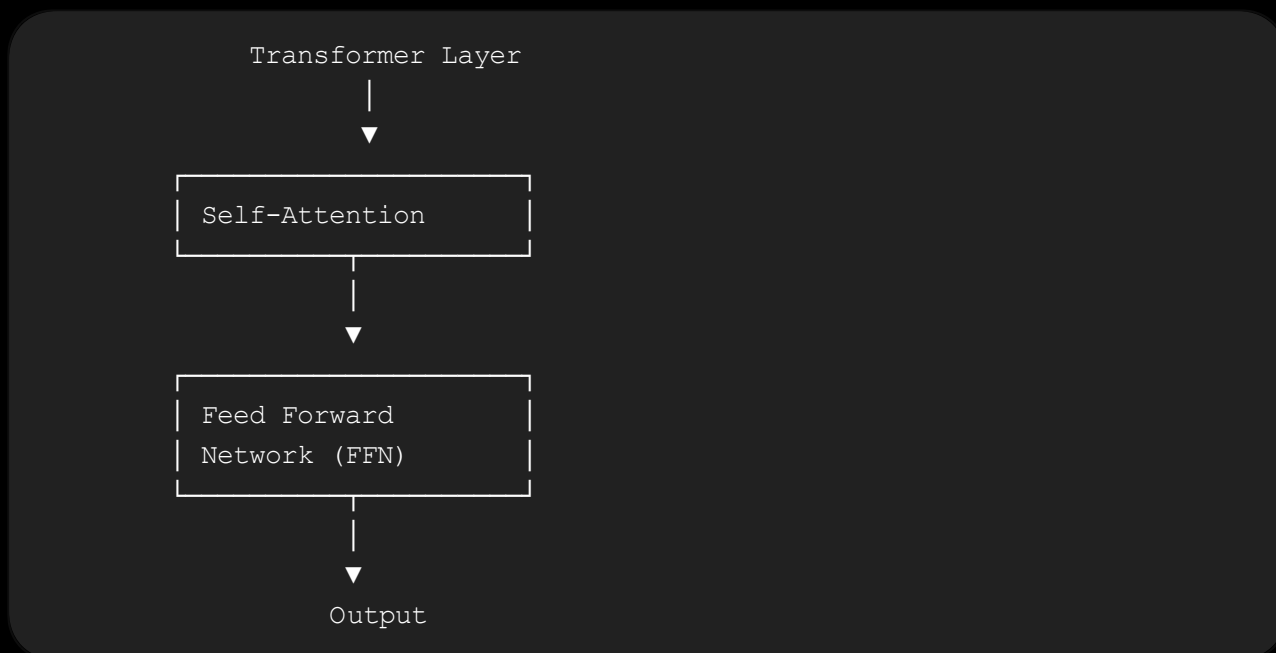
"This is a programming question, therefore use the programming expert."

Instead, it **learns routing patterns during training**.

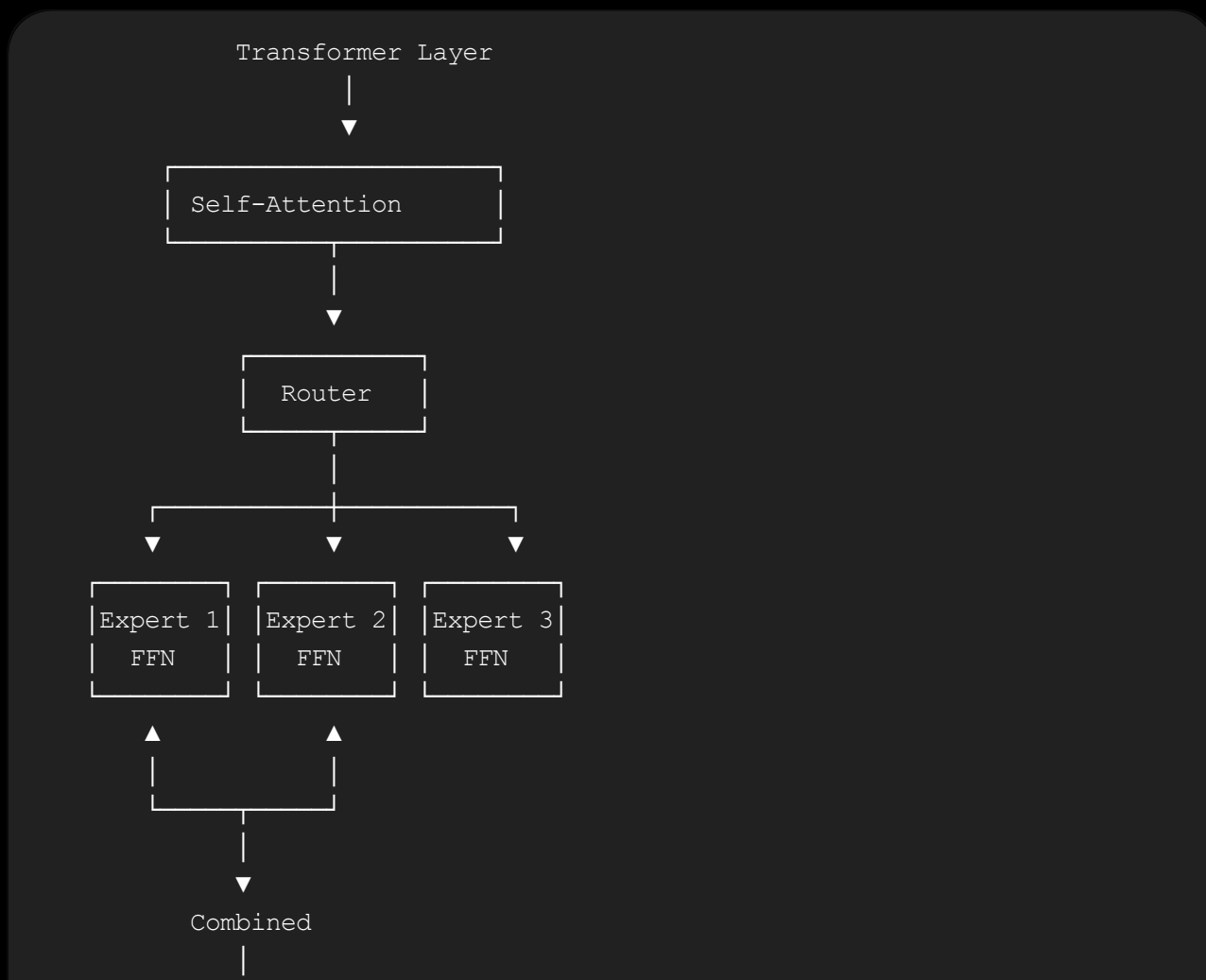
4. MoE inside a Transformer

This is particularly important for understanding modern LLMs.

A Transformer layer traditionally looks roughly like:



In an MoE Transformer, the FFN can be replaced by multiple experts:



▼
Next Transformer
layer

So MoE doesn't replace the Transformer necessarily.

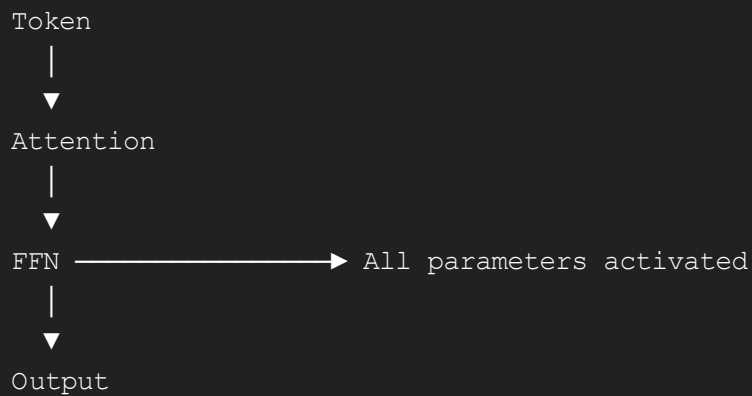
Instead:

MoE can be incorporated inside a Transformer.

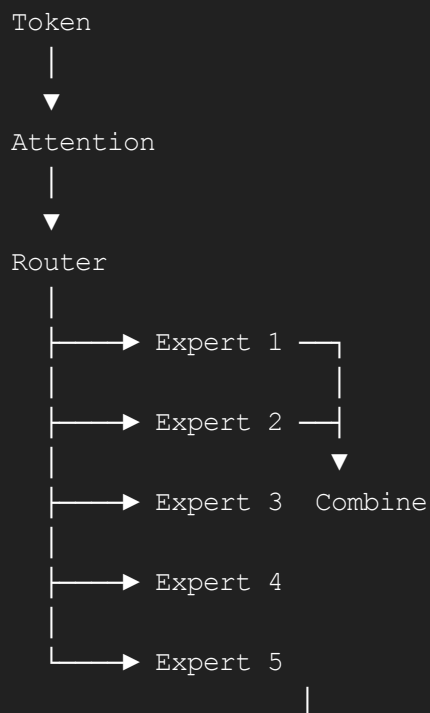
5. Dense Transformer vs MoE Transformer

This is the easiest comparison:

Dense model



MoE model

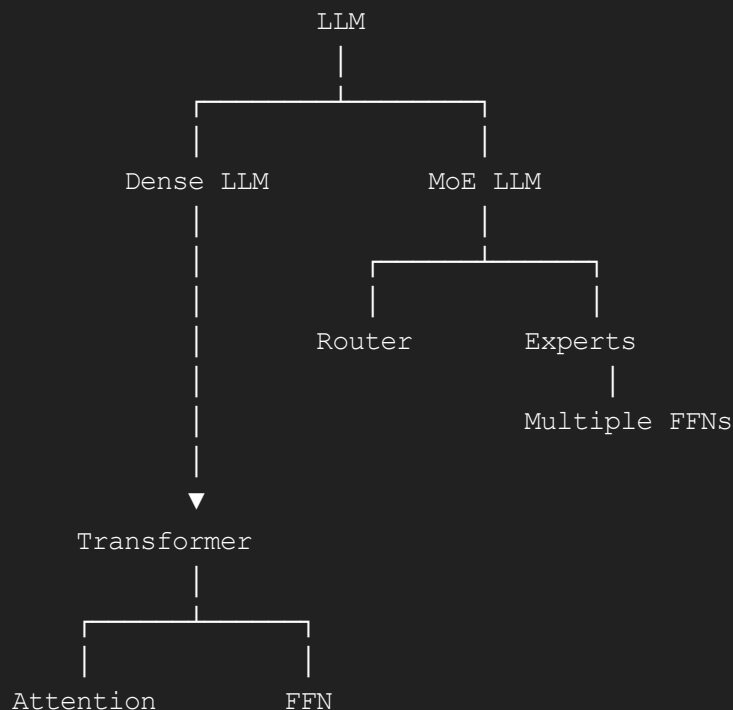


▼
Output

Only selected experts are activated.

6. How does this relate to LLMs?

This gives you a useful hierarchy:



Examples of well-known MoE model families include **Mixtral** and **OpenAI's gpt-oss models**. The important qualification is that the exact implementation differs by model; "MoE" describes the routing/expert strategy rather than one single fixed architecture.

7. How does this affect an Agent?

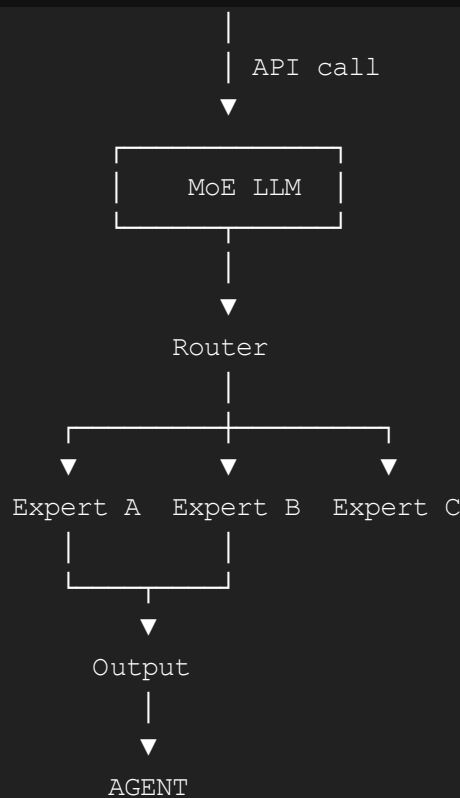
This is where it connects directly to your previous question.

The agent doesn't normally do:

Agent
└─ "I'll select Expert 7."

Instead:

AGENT



Agent chooses the model.

The MoE model's router chooses the experts.

That's a very important distinction.

One-line mental model

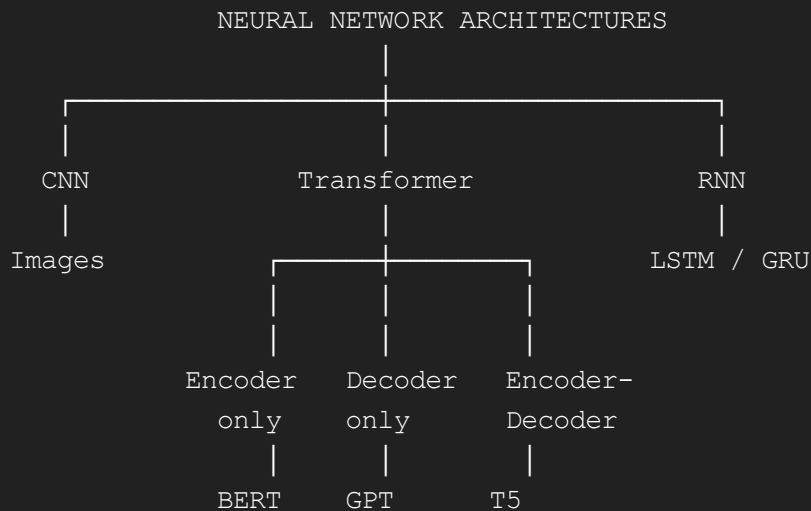
Transformer = overall neural architecture; MoE = a way of giving that architecture multiple specialized expert networks with a learned router that activates only some of them.

Is the understanding correct there are multiple different architectures to call neural networks but these are the important ones? transformer, Encoder only network, Encoder Decoder, CNN.

Yes — **your understanding is broadly correct**, with one important refinement:

Transformer, Encoder-only, Encoder–Decoder, and CNN are important architectures/constructions, but "Encoder-only" and "Encoder–Decoder" are actually variants of the Transformer architecture, not completely separate families.

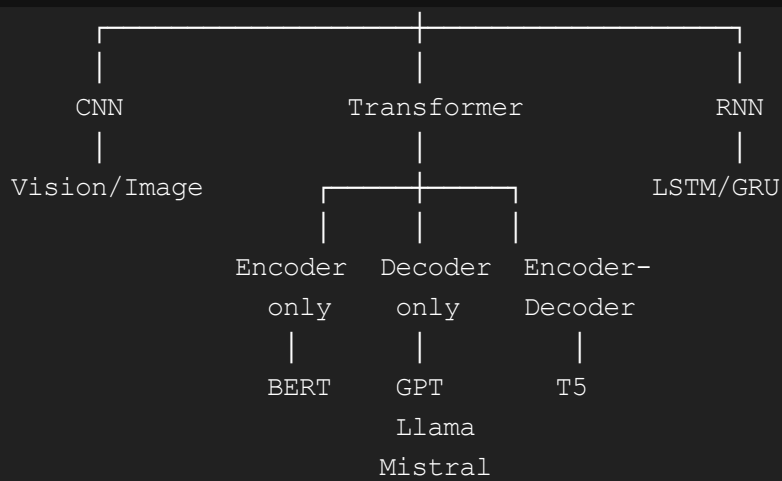
A better mental model is:



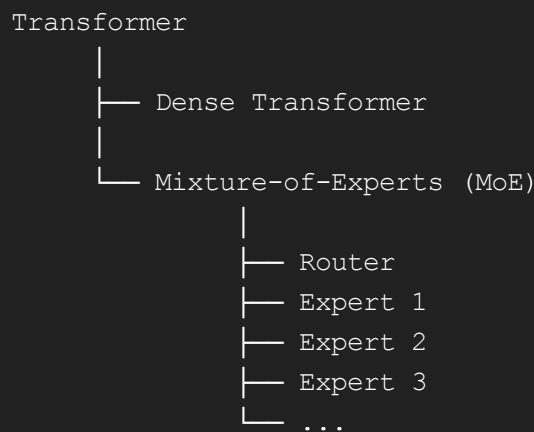
So your list should be understood as:

What you mentioned	More precise classification	Example
Transformer	Neural-network architecture family	Transformer
Encoder-only	Transformer variant	BERT
Encoder-Decoder	Transformer variant	T5, original Transformer
Decoder-only	Transformer variant	GPT-style models, Llama
CNN	Separate neural-network architecture family	ResNet, EfficientNet
RNN	Separate neural-network architecture family	RNN, LSTM, GRU
GNN	Separate architecture family	Graph neural networks
Autoencoder	Network architecture/design pattern	VAE, traditional autoencoders
GAN	Generative architecture/framework	GAN, StyleGAN
Diffusion	Generative modeling architecture/framework	Stable Diffusion

For your Agentic AI learning, I'd organize it this way



Then you have another concept **inside some of these architectures**:



The most important thing for an AI/Agent architect

You don't need to memorize dozens of architectures initially. Understand these layers of abstraction:

1. Neural-network families

CNN, RNN, Transformer, GNN, etc.

2. Transformer configurations

Encoder-only, Decoder-only, Encoder-Decoder

3. Model

BERT, GPT-style models, Llama, T5, etc.

4. Model techniques

MoE, attention variants, quantization, fine-tuning, etc.

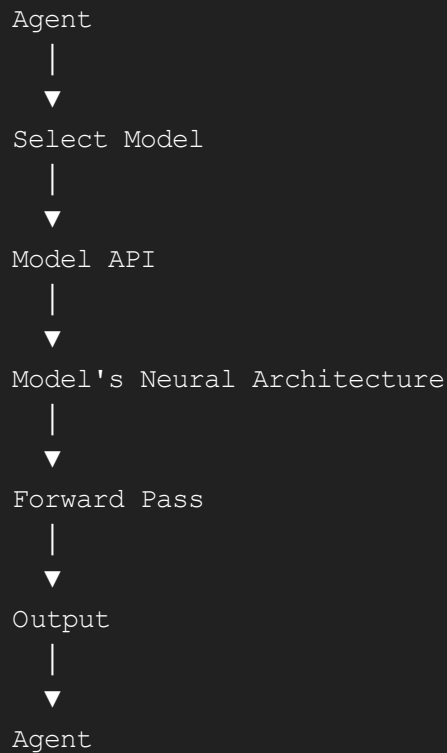
5. Agent

Chooses models/tools, orchestrates calls, maintains state, executes workflows.

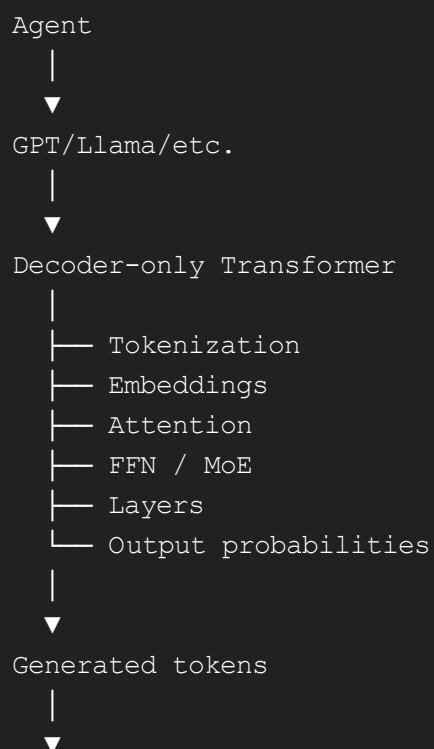
So when you say:

"An agent calls a neural network."

the more technically accurate flow is:



And for a typical LLM agent:



That is the conceptual foundation I'd recommend using when learning Agentic AI architecture.

ChatGPT can make mistakes. Check important info.